

Documentation évaluation

Auteur: Widad

March 13, 2026

Abstract

Documentation du travail effectué pour l'évaluation de réponses. Dernier edit: 12/03/2026

1 évaluation des réponses

Pour le moment on fait une évaluation basée sur deux approches:

- évaluation humaine : utile/partiellement utile/inutile. Juge uniquement l'utilité de la réponse.
- évaluation basée sur des critères de performance: rapidité, consommation de ressources, absence d'erreurs.

1.1 evaluer_reponses.py

C'est le script principal qui évalue les tickets. Il contient aussi une fonction de comparaison des différents modèles.

A l'exécution du fichier 2 choses se produisent:

- Il génère le fichier tickets_evalues_scores.json.
- Il affiche également dans le terminal une comparaison des différents modèles en se basant sur les tickets correspondants.

1.2 tickets_evalues_fake.json

Base de tickets fictifs et leurs évaluations également fictives.

Variables retenues:

- id_ticket
- contexte
- ticket
- modele

- dateheure
- reponse
- feedback_utilisateur
- temps_reponse (s)
- nombre_tokens

Les améliorations à faire ici: il faut mettre à jour les noms de modèles si d'autres que ceux-ci sont utilisés + faire un programme qui génère automatiquement ce fichier à partir de vrais (tickets,réponses) + il faut faire le lien avec `evaluation_manager.py` qui récupère les feedbacks utilisateur (ajouter une option "moyennement utile" ici) + également il faut remplacer "ERREUR" par les vrais messages d'erreur qu'on obtient qu'on l'agent se plante (éventuellement définir un tel message si pas encore fait).

1.3 `config_evaluation.py`

Ce fichier est en gros la formule qu'on utilise pour calculer le score qui va être attribué à chaque ticket évalué. Il peut être édité à volonté par exemple pour faire varier le poids attribué à chaque variable. Pour cette même raison c'est un fichier séparé du code d'évaluation.

Pour le moment on part sur 4 variables:

- erreur_penalty (si erreur, on pénalise avec un score de 0)
- pertinence (feedback utilisateur: utile, pas utile, moyennement utile)
- vitesse (temps de réponse)
- efficacité (nombre de tokens)

Ensuite on combine ces 4 critères avec des poids différents pour obtenir un score par ticket. J'ai fait le choix (modifiable) de prendre 35% pertinence, 25% pénalité erreur, 20% vitesse, et 20% efficacité. Ainsi la pertinence est le critère le plus important, les erreurs comptent, la vitesse et l'efficacité comptent un peu moins.

Pour calculer les sous_scores pour chaque critère (vitesse, pertinence etc) le feedback textuel est converti en valeurs numériques, et de même les valeurs temporelles (continues) relatives à la vitesse sont converties en scores suivant des seuils (par exemple: moins d' 1.0s = excellent, plus de 30.0 = lent, etc).

1.4 `tickets_values_scores.json`

Ici on stocke les scores et sous-scores attribués aux tickets.

2 La suite

- On pourra améliorer le volet évaluation humaine en ajoutant d'autres aspects comme le ton de la réponse, l'exactitude de la réponse (par exemple pour une question dont la réponse est déjà connue).
- On peut faire évaluer les réponses par un modèle plus puissant, avec par exemple une grille de critères.
- On peut demander à l'humain ou au modèle de choisir la meilleure parmi deux réponses (générées par deux modèles différents) (par exemple pour définir le meilleur modèle pour un domaine particulier).

3 bibliographie

- [1] <https://blog.jetbrains.com/pycharm/2024/08/how-to-build-chatbots-with-langchain/#step-1-project-creation,-prerequisites,-and-required-library-installation>
- [2] <https://hyperskill.org/courses/149-ai-agents-theory-and-practice>